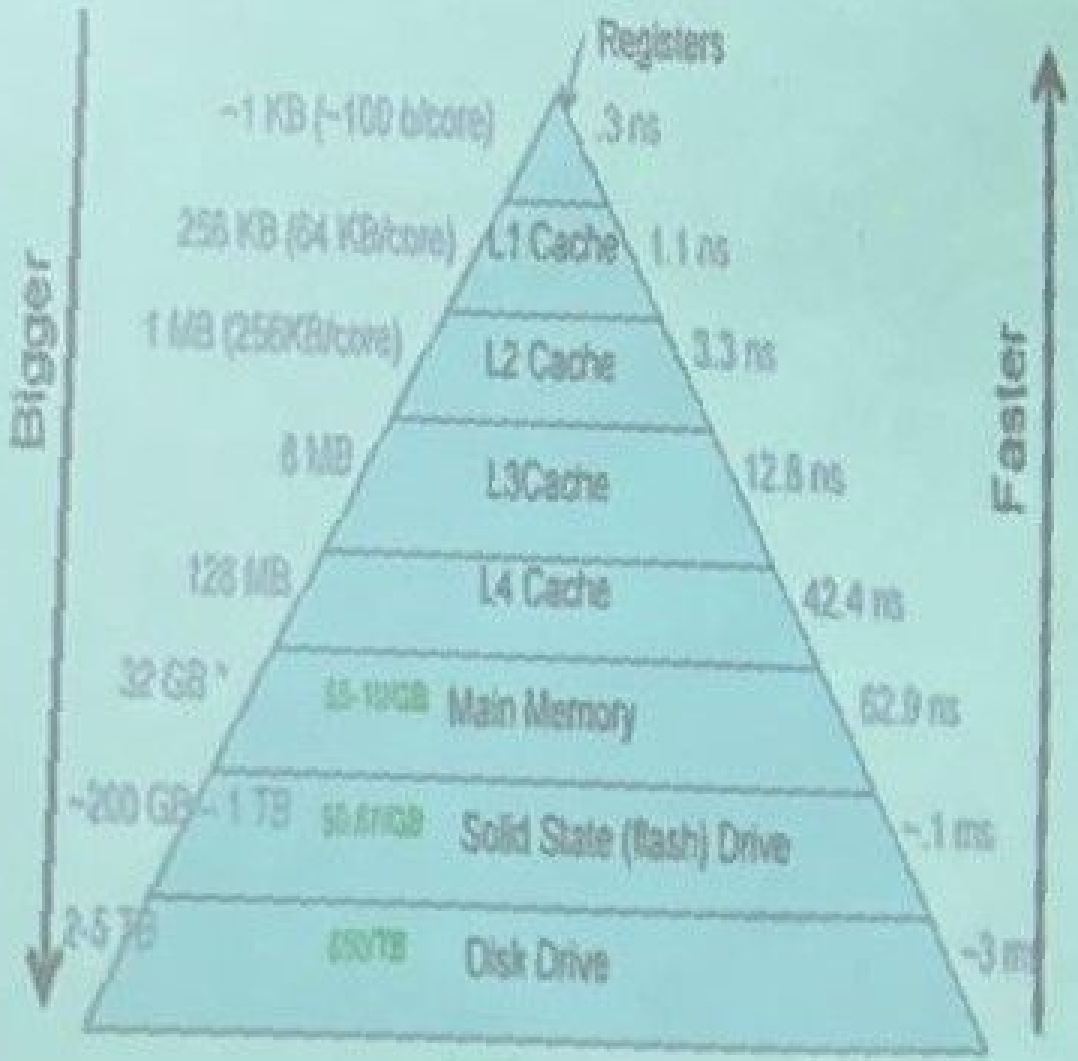
Elements of Cache Design

- Size
  - Cache Size,
  - Block Size
  - Line Size
- Addressing
  - Logical
  - Physical
- Mapping Function
  - Direct,
  - Associative,
  - Set Associative
- Write Policy
  - Write through,
  - Write back,
  - Write once
- Cache Organization
  - Single or Multi level,
  - Unified or Split
- Replacement Algorithms
  - Least recently used (LRU),
  - First in first out (FIFO),
  - Least frequently used (LFU),
  - Random

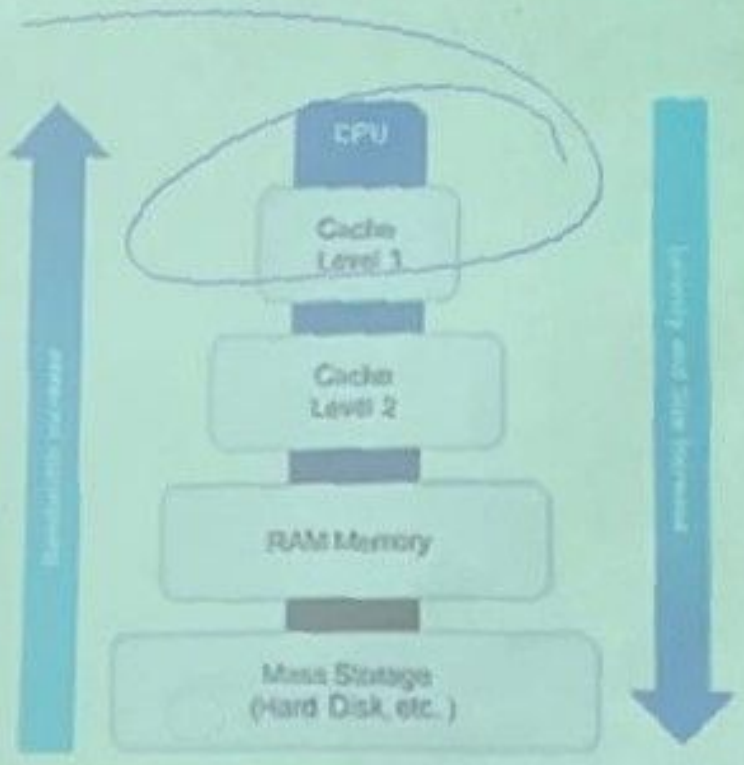
Cache Size

- Cache memory is intended to give memory speed approaching that of the fastest memories available.
- At the same time it provides a large (sufficient) memory size at the price of less expensive types of semiconductor memories.
- The size of the cache to be small enough so that the overall average cost per bit is close to that of main memory alone and large enough so that the overall average access time is close to that of the cache alone.



Cache Size Matter !!!

- The size of the cache must be small enough so that the overall average cost per bit is close to that of main memory alone and large enough so that the overall average access time is close to that of the cache alone.
- Cache size does matter for
  a) Cost : More cache is expensive
  b) Speed
     i. More cache is faster (up to a point)
     ii. Checking cache for data takes time



Comparison of Cache Sizes

Processor	Type	Year of Introduction	L1 cache	L2 cache	L3 cache
IBM 360/85	Mainframe	1968	16 to 32 KB	—	—
PDP-11/70	Minicomputer	1975	1 KB	—	—
VAX 11/780	Minicomputer	1978	16 KB	—	—
IBM 3033	Mainframe	1978	64 KB	—	—
IBM 3090	Mainframe	1985	128 to 256 KB	—	—
Intel 80486	PC	1989	8 KB	—	—
Pentium	PC	1993	8 KB/8 KB	256 to 512 KB	—
PowerPC 601	PC	1993	32 KB	—	—
PowerPC 620	PC	1996	32 KB/32 KB	—	—
PowerPC G3	PC/server	1999	32 KB/32 KB	256 KB to 1 MB	2 MB
IBM S/390 G4	Mainframe	1997	32 KB	256 KB	2 MB
IBM S/390 G6	Mainframe	1999	256 KB	8 MB	—
Pentium 4	PC/server	2000	8 KB/8 KB	256 KB	—
IBM SP	High-end server/ supercomputer	2000	64 KB/32 KB	8 MB	—
CRAY MTA	Supercomputer	2000	8 KB	2 MB	—
Itanium	PC/server	2001	16 KB/16 KB	96 KB	4 MB
SGI Origin 2001	High-end server	2001	32 KB/32 KB	4 MB	—
Itanium 2	PC/server	2002	32 KB	256 KB	6 MB
IBM POWER5	High-end server	2003	64 KB	1.9 MB	36 MB
CRAY XD-1	Supercomputer	2004	64 KB/64 KB	1 MB	—

Comparison of Cache Sizes

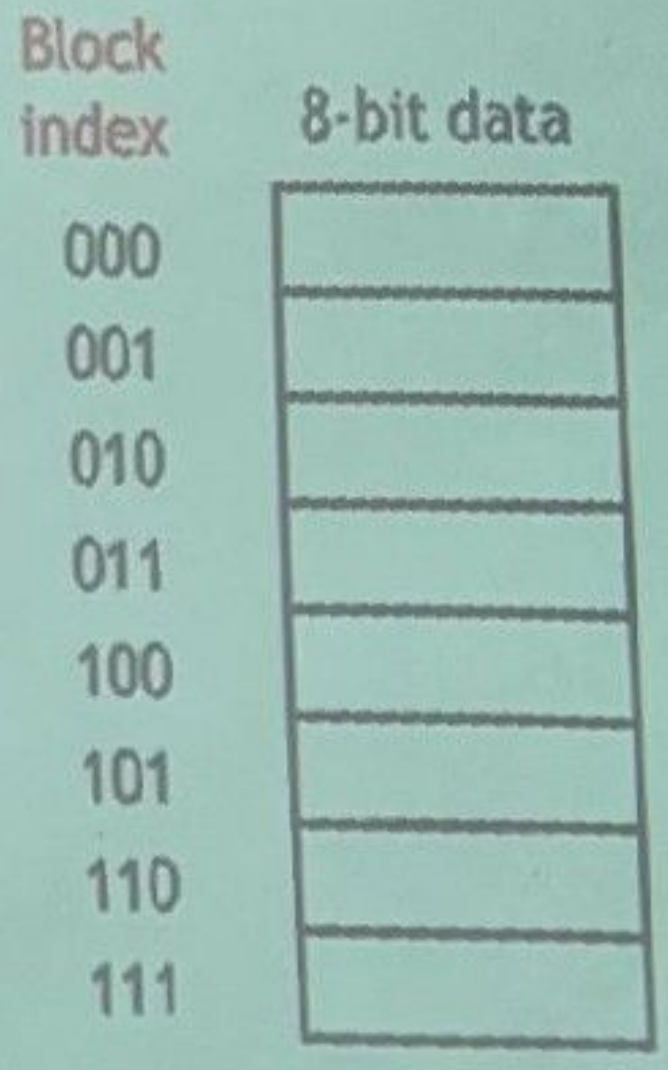
IBM POWER5	High-end server	2003	64 kB	1.9 MB	36 MB
CRAY XD-1	Supercomputer	2004	64 kB/64 kB	1MB	—
IBM POWER6	PC/server	2007	64 kB/64 kB	4 MB	32 MB
IBM z10	Mainframe	2008	64 kB/128 kB	3 MB	24-48 MB
Intel Core i7 EE 990	Workstation/ server	2011	6 × 32 kB/32 kB	1.5 MB	12 MB
IBM zEnterprise 196	Mainframe/ Server	2011	24 × 64 kB/ 128 kB	24 × 1.5 MB	24 MB L3 192 MB L4

Cache Unit

- Cache Size,
- Block Size
- Line Size

Cache Block

- Caches are divided into blocks, which may be of various sizes.
- The basic unit for cache storage.
- May contain multiple bytes/words of data.
  - The number of blocks in a cache is usually a power of 2.
  - For now we'll say that each block contains one byte.
- Here is an example cache with eight blocks, each holding one byte.



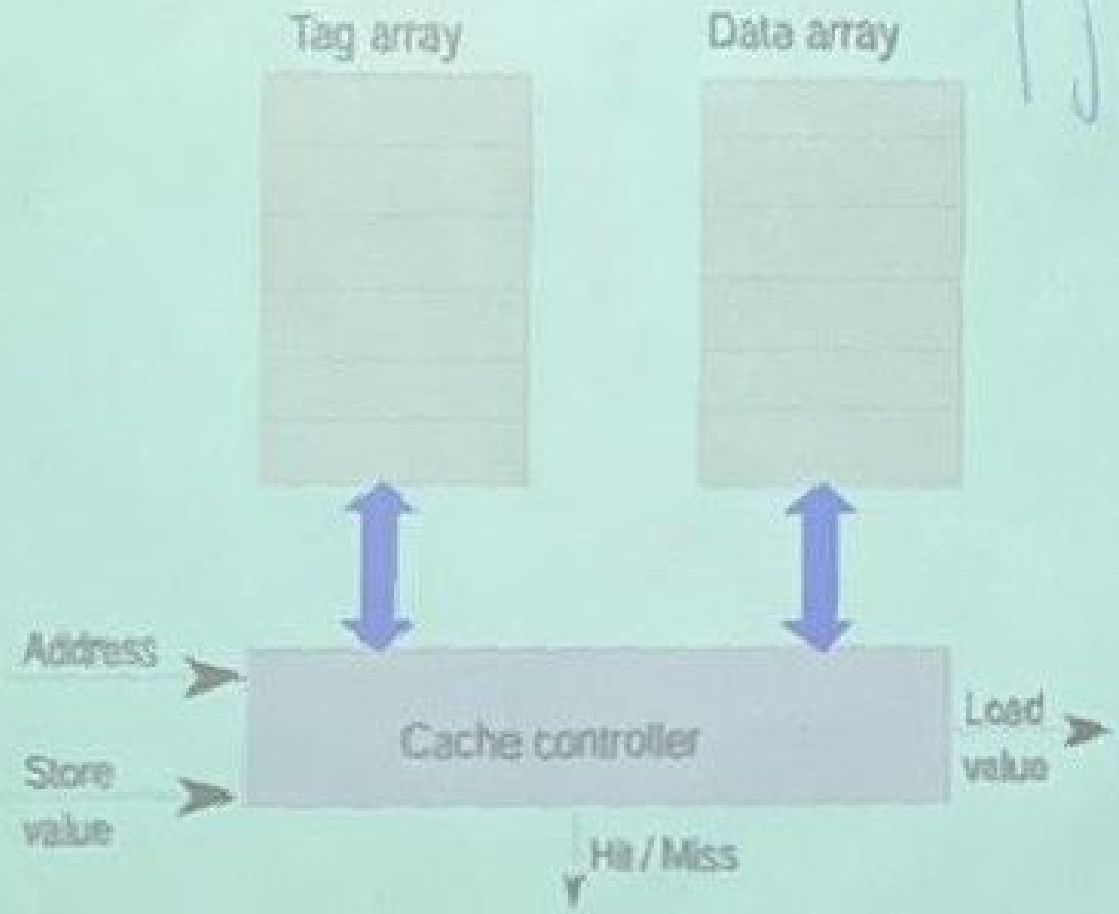
Cache Line

- The chunks of memory handled by the cache are called cache lines.
- The size of these chunks is called the cache line size.
- Same as cache block.
- **Note that this is not the same thing as a "row" of cache.**
- Common cache line sizes are 32, 64 and 128 bytes.
- A cache can only hold a limited number of lines, determined by the cache size.
- For example, a 64 kilobyte cache with 64-byte lines has 1024 cache lines.

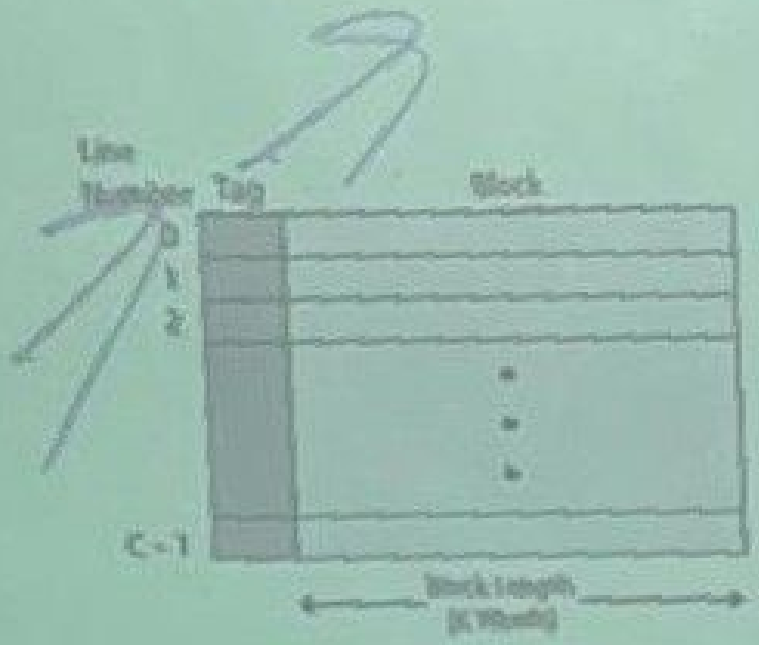
Cache Set, Tag, and Valid Bit

- **Cache set**
  - A "row" in the cache.
  - The number of blocks per set is determined by the layout of the cache
  - Determines direct mapped, set-associative, or fully associative cache.
- **Tag**
  - A unique identifier for a group of data.
  - Because different regions of memory may be mapped into a block,
  - the tag is used to differentiate between them.
- **Valid bit**
  - A bit of information that indicates whether the data in a block is valid (1) or not (0).

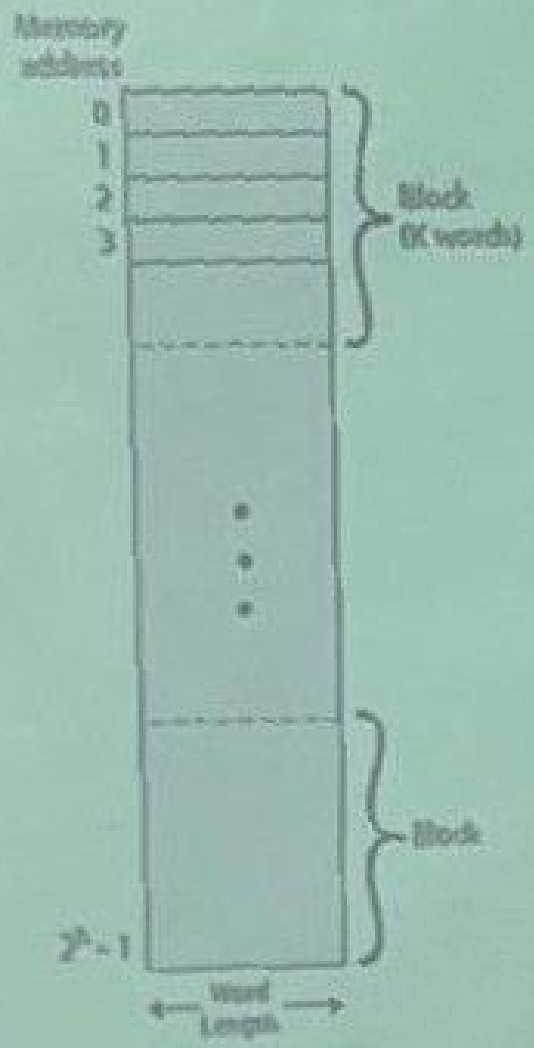
Structure of a Cache



Cache/Main Memory Structure



(a) Cache



(b) Main memory

Cache/Main Memory Blocks

- Cache of 64 kByte
- Cache block of 4 bytes
  - i.e. cache is 16k (2^{14}) lines of 4 bytes
- 16 MBytes main memory
- 24 bit address
  - (2^{24}=16M)

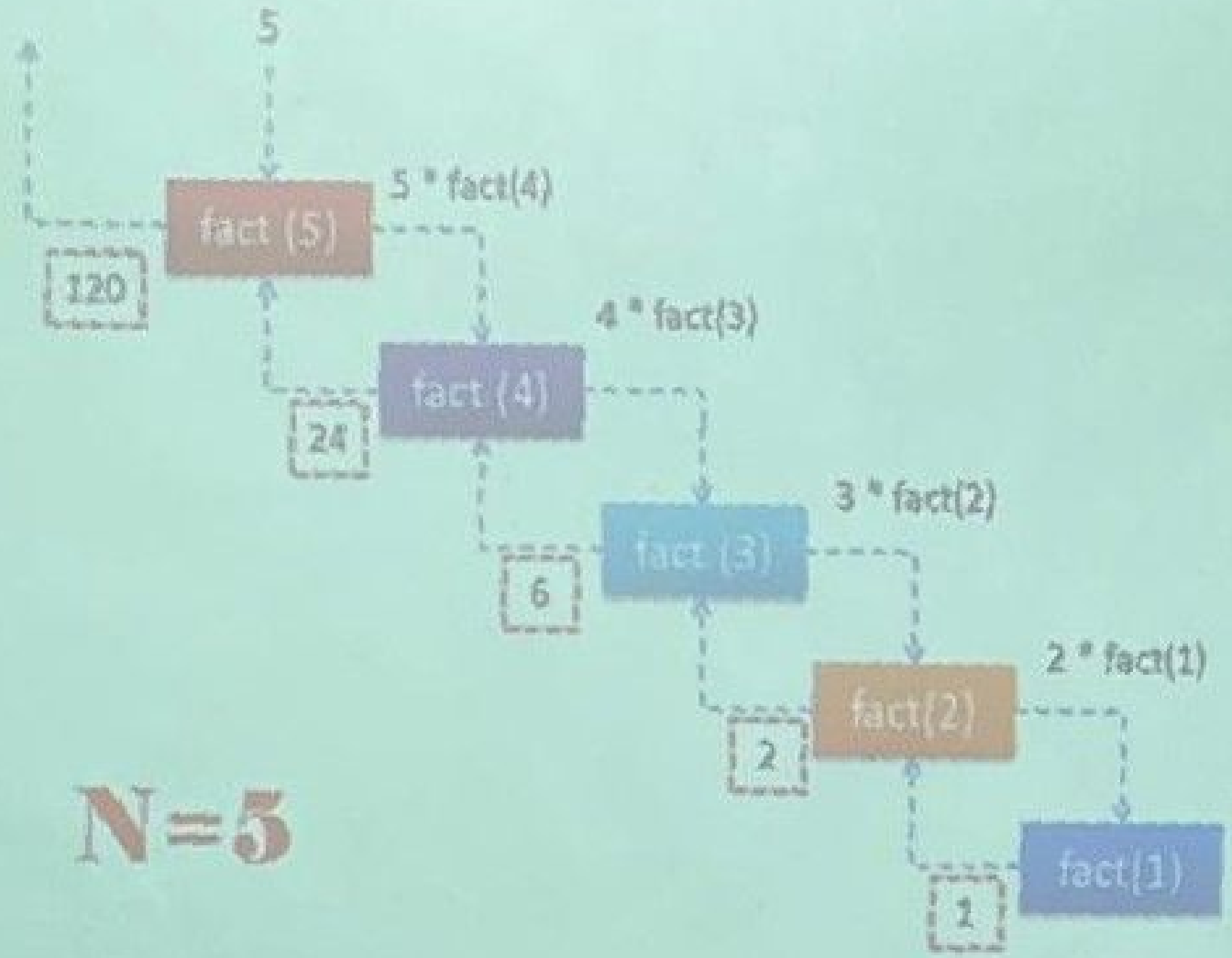
Cache Addressing

- Physical Addressing
- Logical Addressing

Cache Size Constraints

- Cache size design constraints
  - Cost
  - Speed/Access Time

Factorial (N)



N=5

N! !!!!!!!

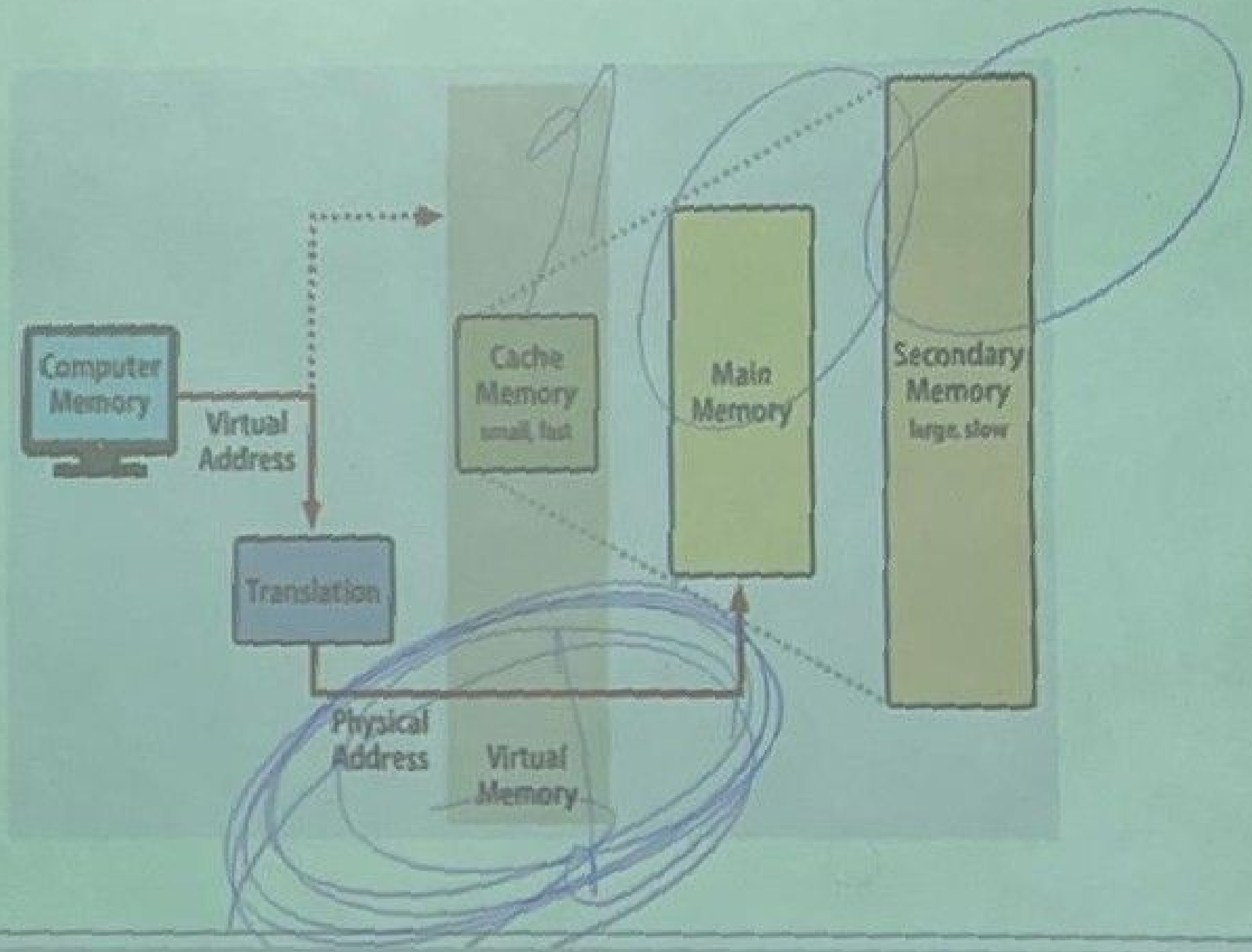
100! = 93 326 215 443 944 152 681
699 238 856 266 700 490 715 968
264 381 621 468 592 963 895 217
599 993 229 915 608 941 463 976
156 518286 253 697 920 827 223
758 251 185 210 916 864 000 000
000 000 000 000 000 000

n	n!
1	1
2	2
3	6
4	24
5	120
6	720
7	5040
8	40320
9	362880
10	3628800
11	39,916,800
12	479,001,600
13	6,227,020,800
14	87,178,291,200
15	1,307,674,368,000
16	20,922,789,888,000

Virtual Memory

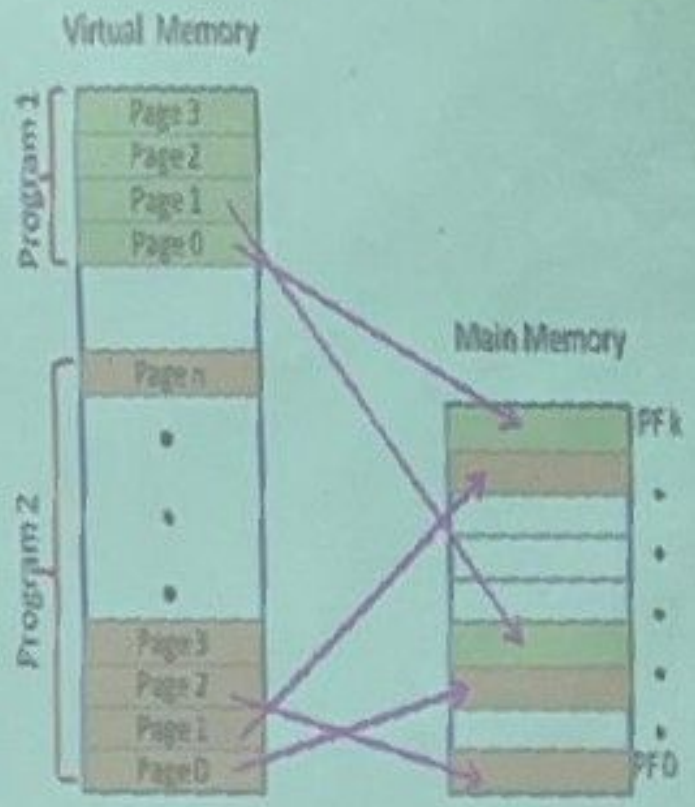
- How to handle larger program over small cache?
  - Virtual Memory
- Many processors support **Virtual Memory**.

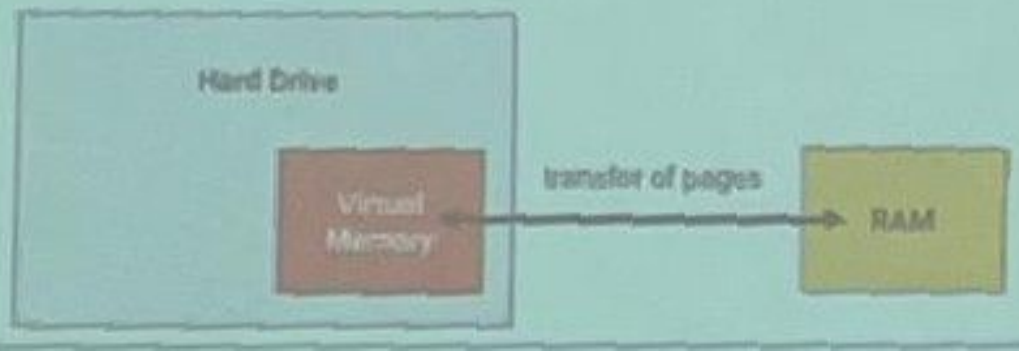
Virtual Memory



Virtual Memory

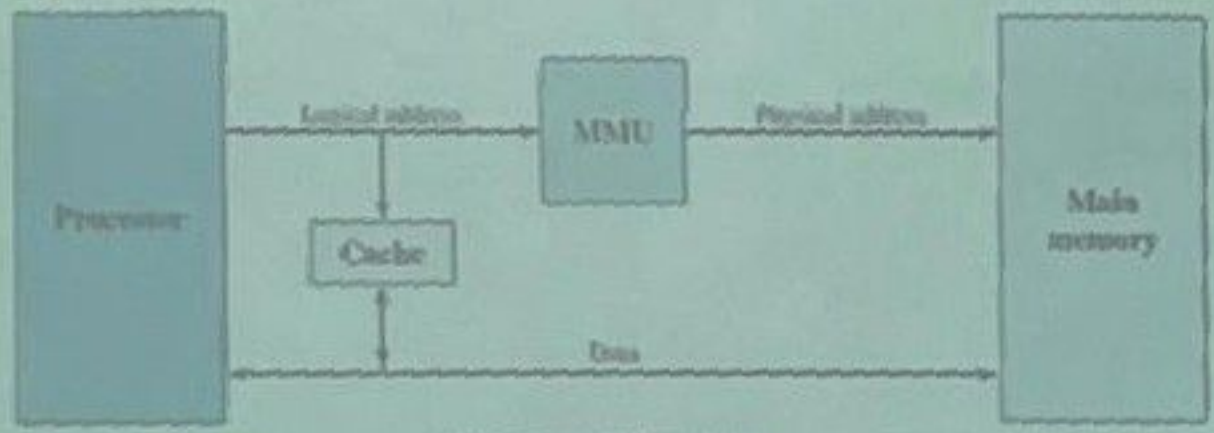
- Virtual memory is a facility that allows programs to address memory from a logical point of view, without regard to the amount of main memory physically available .
- When virtual memory is used, the address fields of machine instructions contain ***virtual addresses***.
- The hardware device called Memory Management Unit (**MMU**) is used for mapping this logical address to the physical address.



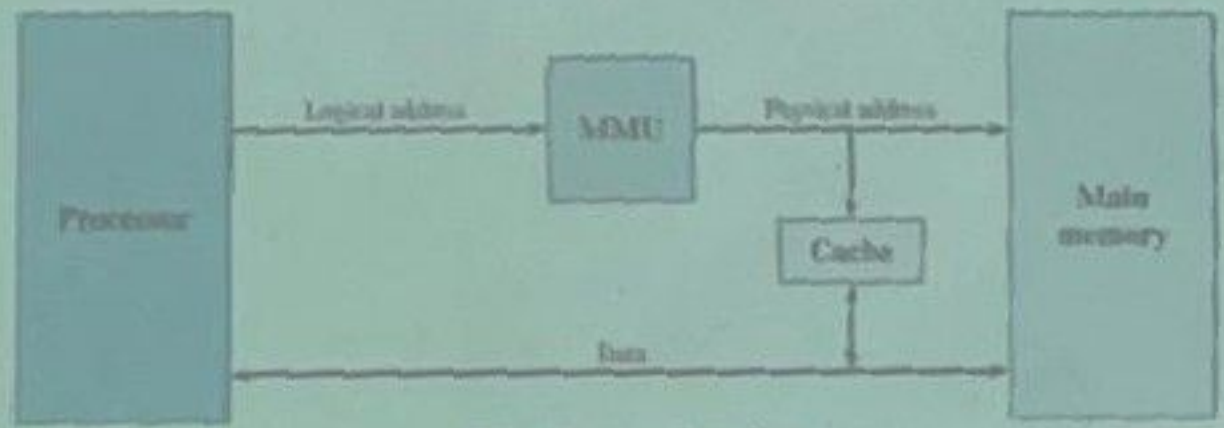


Virtual Memory

- When virtual addresses are used, the system designer may choose to place the cache between the processor and the MMU or between the MMU and main memory.



Logical Cache

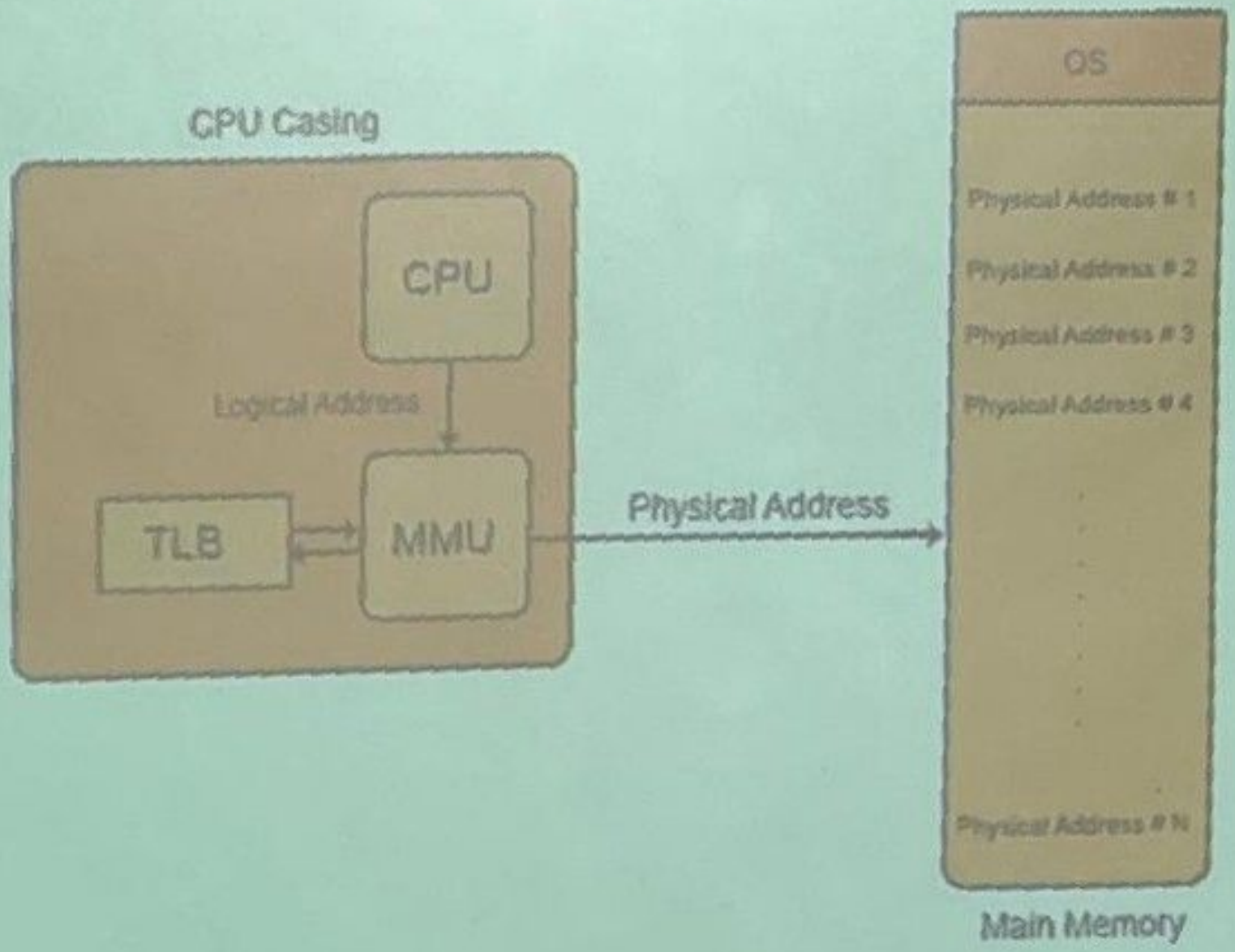


Physical Cache

Physical Address

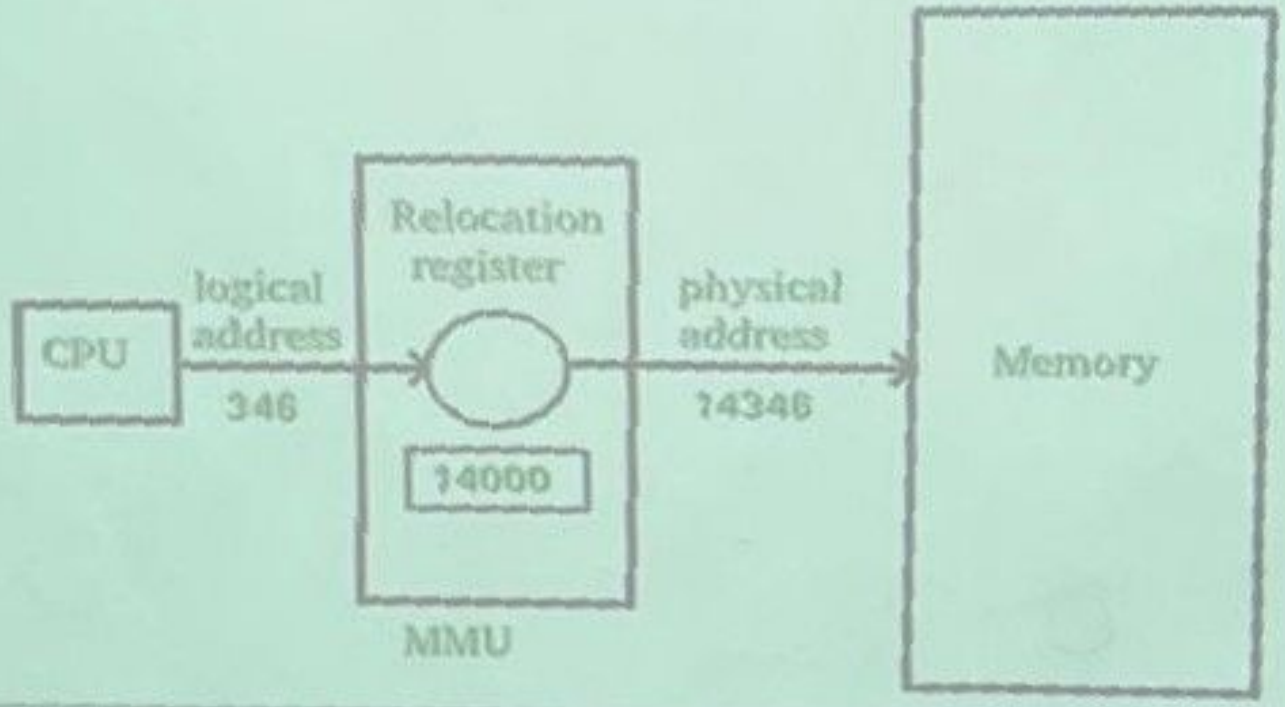
- In operating systems, *logical and physical addresses* are used to manage and access memory.
- Physical Address is the actual address of the data inside the memory.
- A program needs physical memory for its execution.
- The user never deals with the Physical Address.
- The user program generates the logical address and is mapped to the physical address by the Memory Management Unit(MMU).
- The logical address is a virtual address.

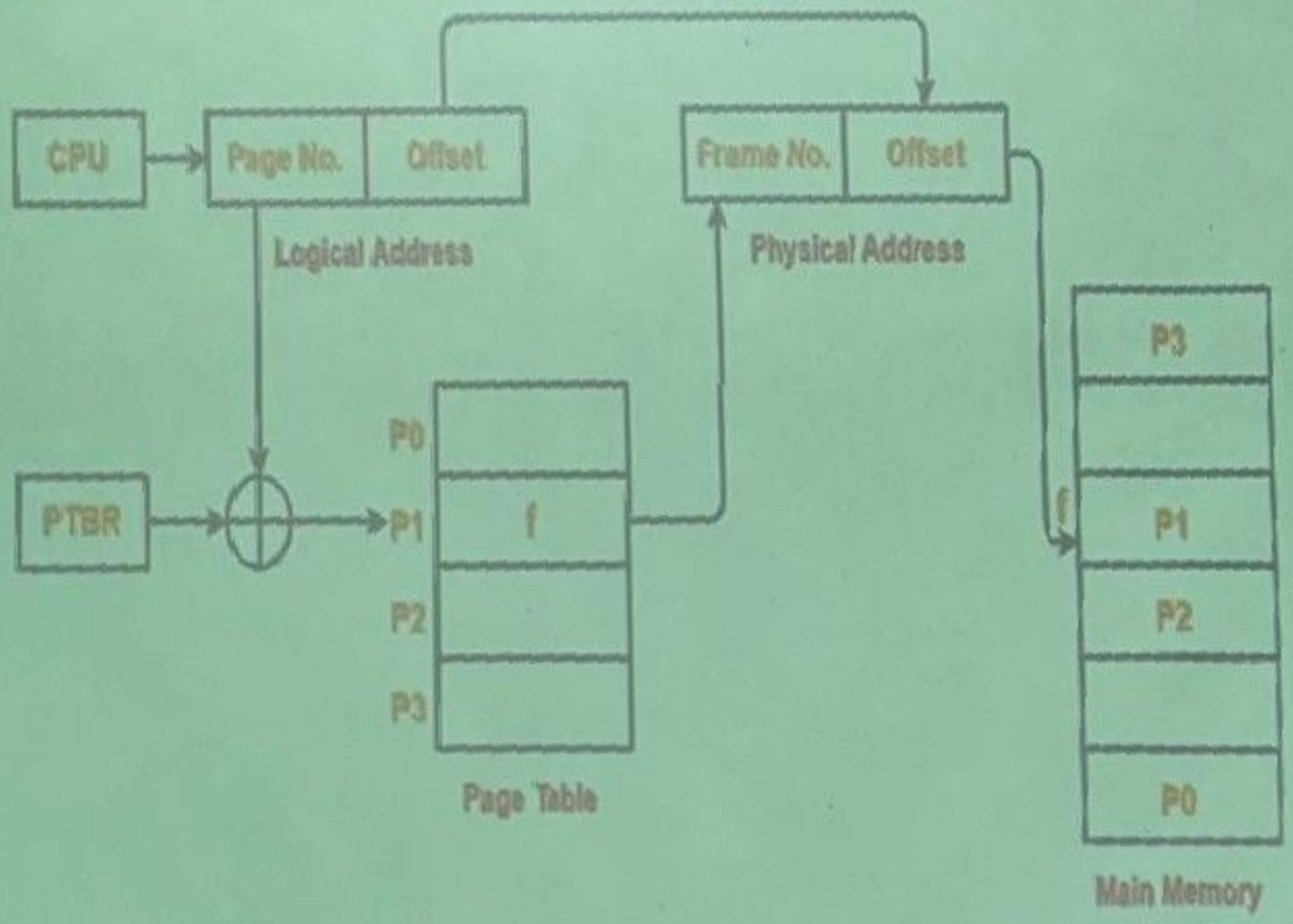
Physical Address



Address Translation

- Address translation is the process of mapping one address type to another (*physical* ←→ *logical*)
- The MMU uses a page table to translate logical addresses into physical addresses.
- The page table maps each logical page number to a physical frame number.

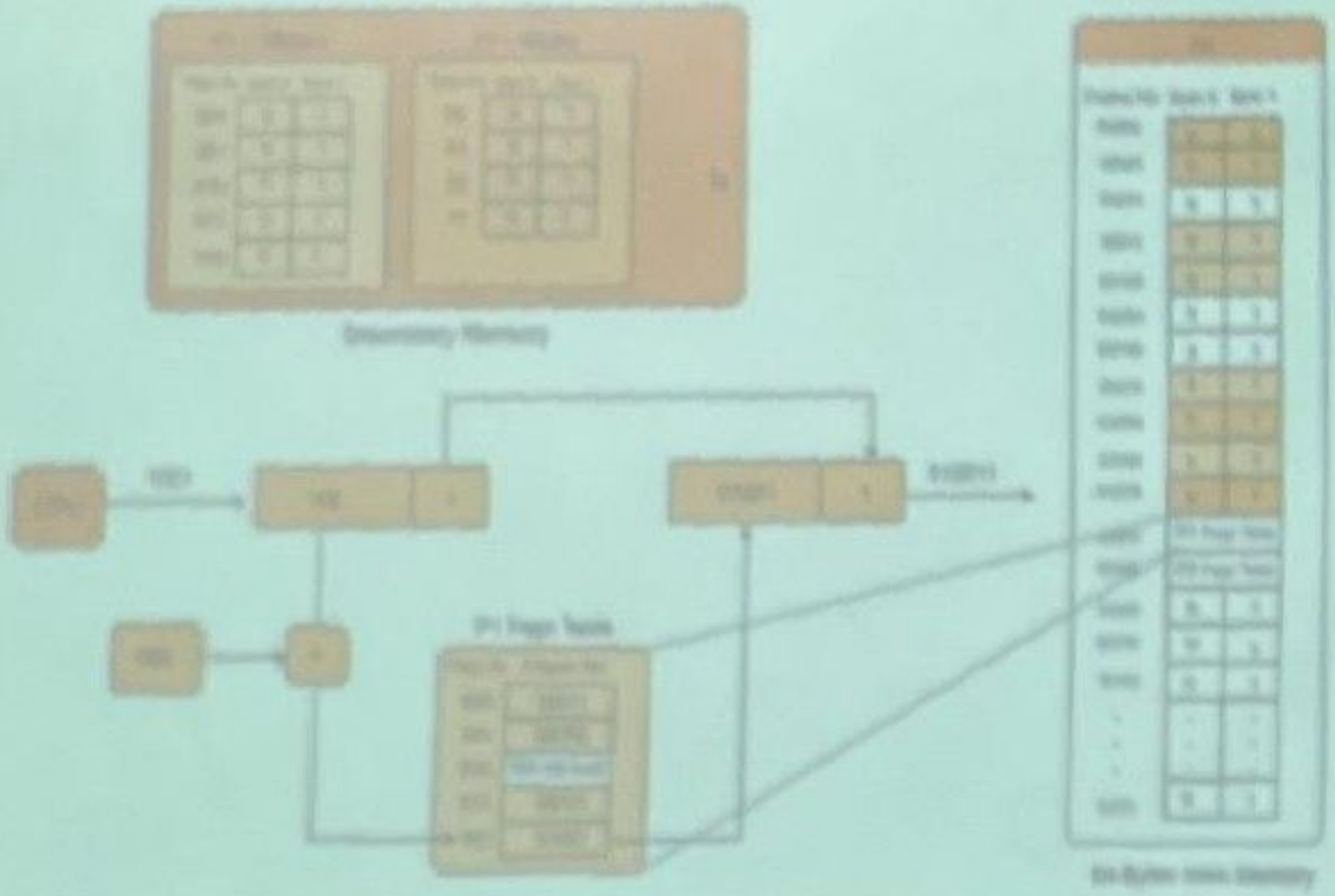




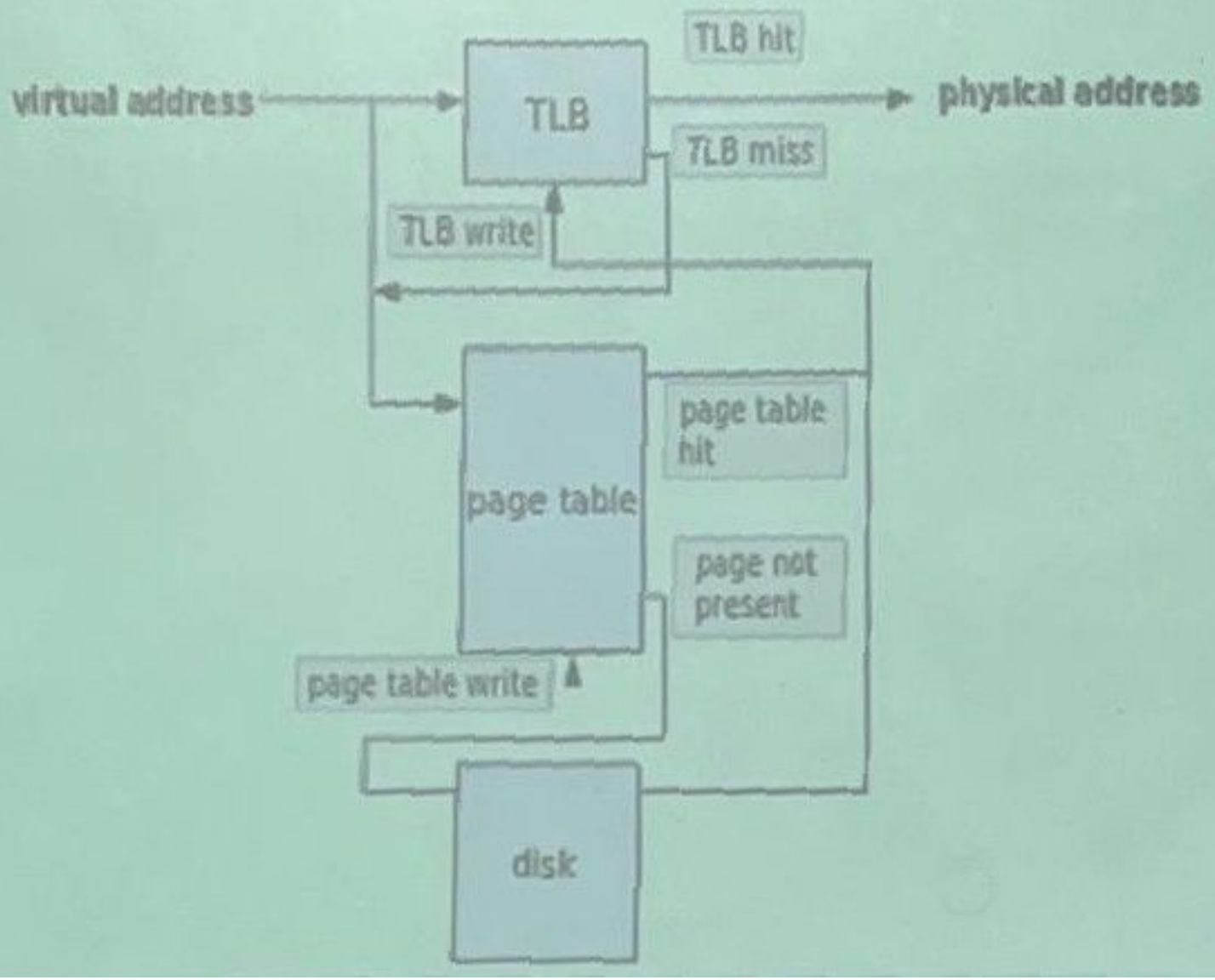
Translating Logical Address into Physical Address

Physical Address

- The set of all physical addresses corresponding to the logical addresses in the logical address space is called the *physical address space*.

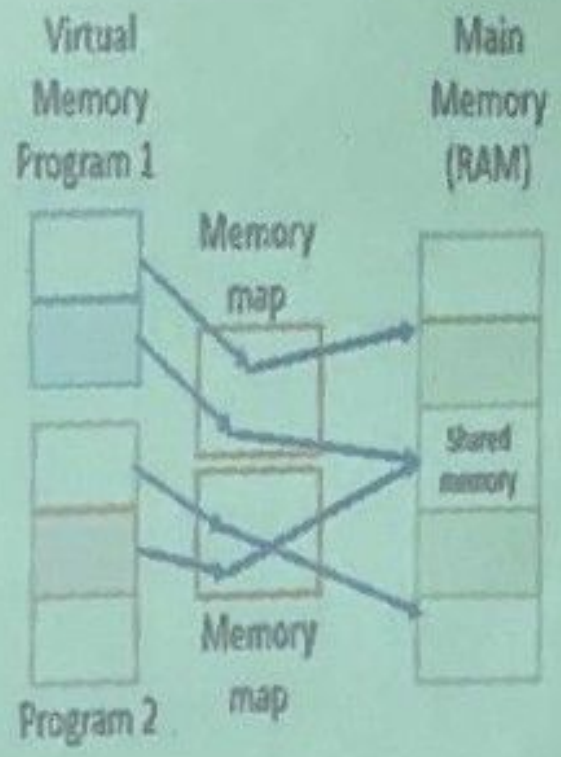


Physical Address



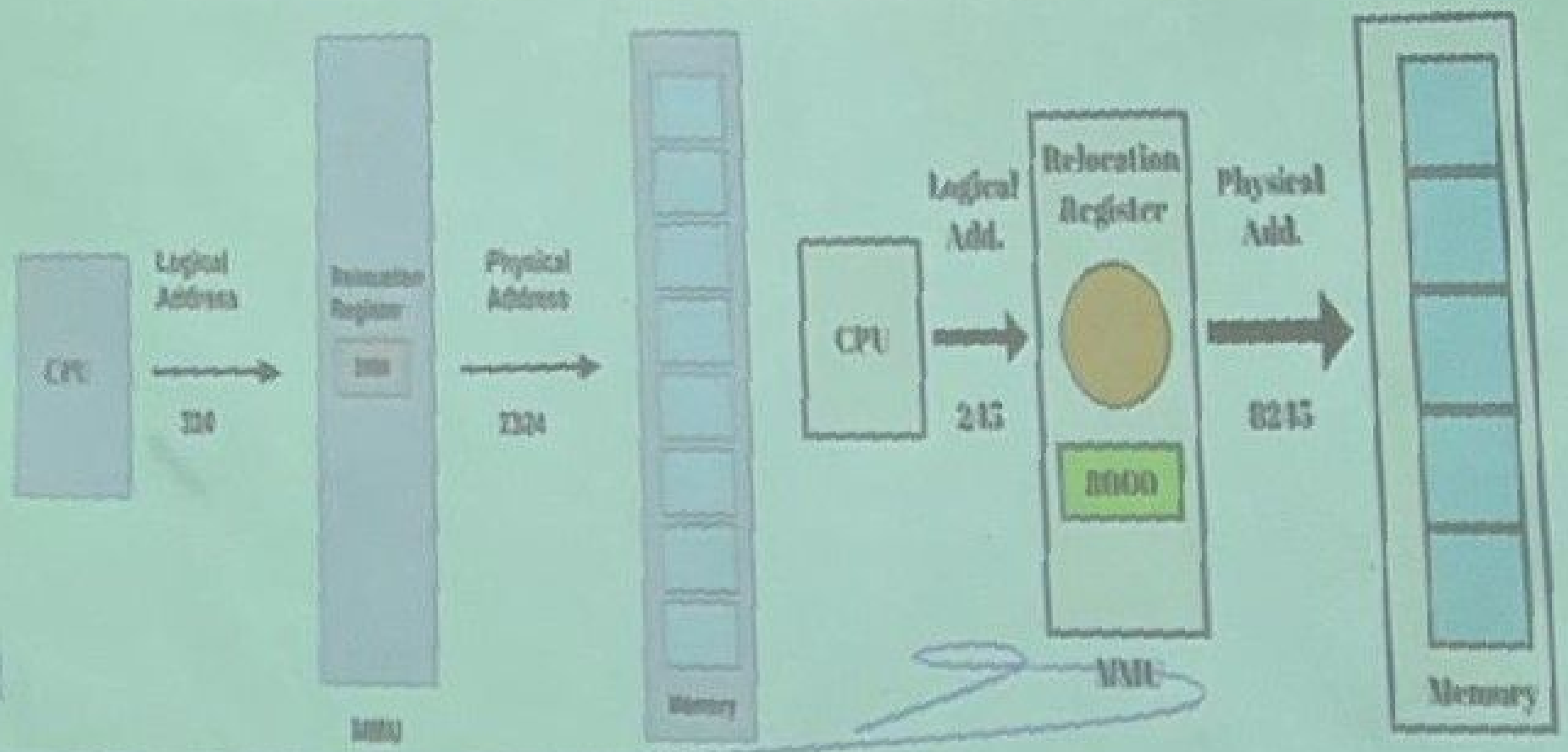
Logical Address

- *A logical address, also known as a virtual address, is an address generated by the CPU during program execution.*
- *It is referred to as a **virtual address** because it does not exist physically.*
- It is the address seen by the process and is relative to the program's address space,
- The process accesses memory using logical addresses, which are translated by the operating system into physical addresses.
- Using this address, the CPU access the actual address or physical address inside the memory, and data is fetched from there.
- *The set of all logical addresses generated by the CPU for a program is called the **logical address space**.*



Logical Address

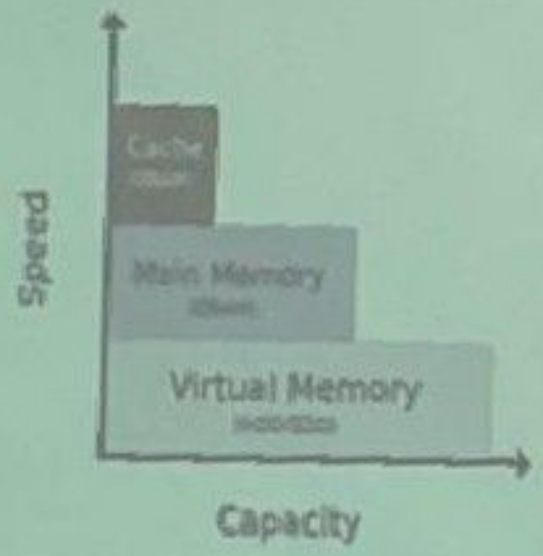
- The hardware device called Memory Management Unit (MMU) is used for mapping this logical address to the physical address.

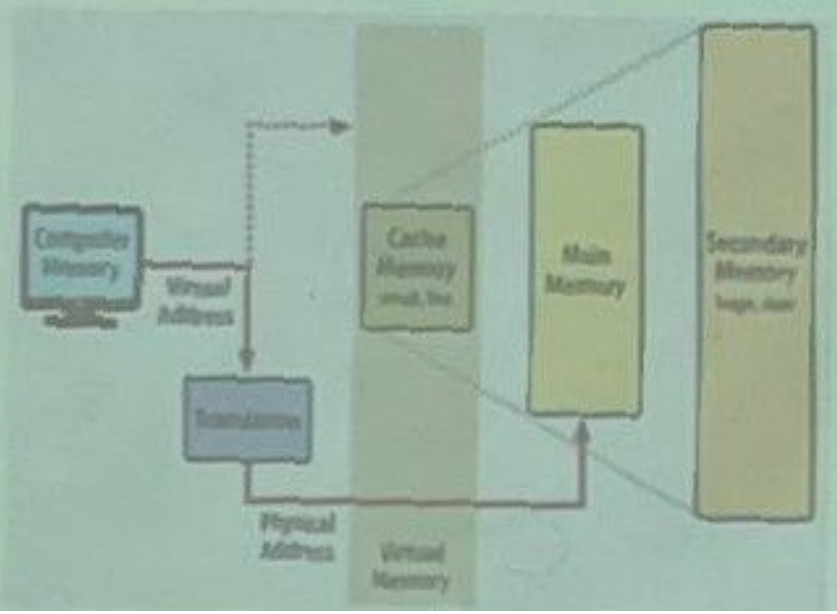


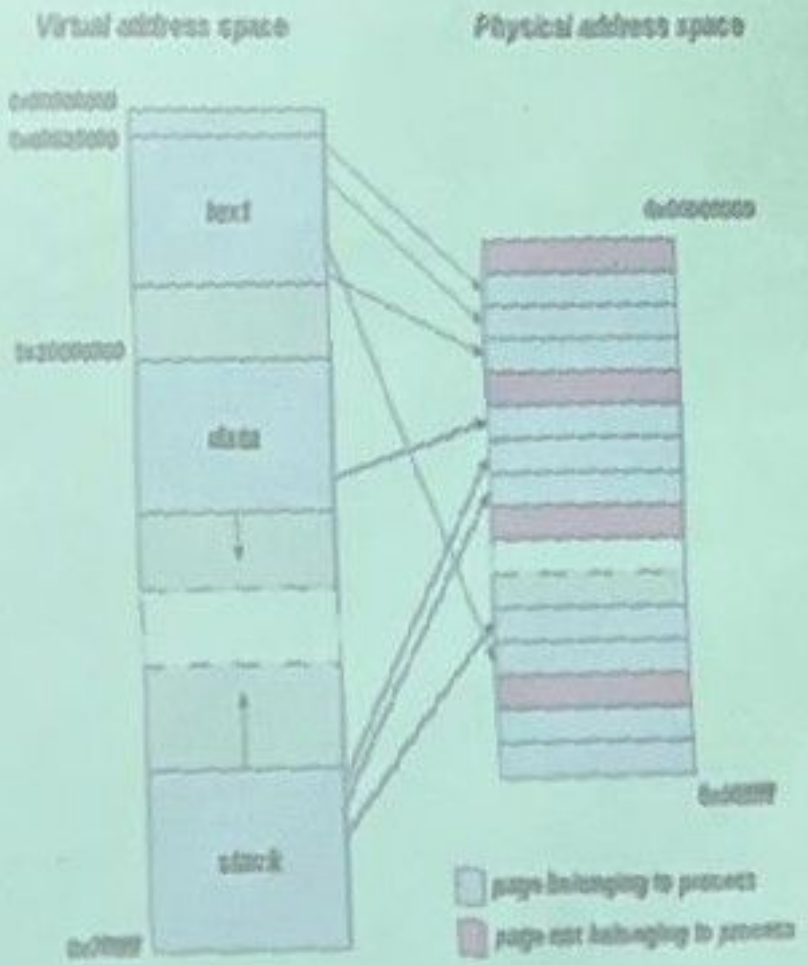
Logical Cache

- Logical cache (virtual cache) stores data using virtual addresses.
- The processor accesses the cache directly, without going through the MMU.

- **Advantage** of the logical cache is that cache access faster than for a physical cache, before MMU address translation





Logical Cache

- **Disadvantage** has to do with the fact that most virtual memory systems supply each application with the same virtual memory address space.
  - That is, each application sees a virtual memory that starts at address 0.
  - Thus, the same virtual address in two different applications refers to two different physical addresses.
  - The cache memory must therefore be completely flushed with each application context switch, or extra bits must be added to each line of the cache to identify with virtual address space this address refers to!